WEEK 7

Data Preparation, Modelling and DAX Measures in Power BI

Case study: Students Performance dataset

Supplementary exercise: SHOP stock price data

J.MOHANA SRI

E25DA027

Table of content:

1. Introduction

2. Dataset Description

3. Data Cleaning and Transformation in Power Query

4. Creating the Students Mark Table (Unpivot)

5. Data Model

6. DAX Measures

7. Observations, Issues and Improvements

8. Learning Outcomes

9. Conclusion

1. Introduction

Raw data is rarely ready for analysis. Before any chart or dashboard can be trusted, the data has to be loaded, checked, cleaned, shaped into a suitable structure and linked together in a model. This report documents the work done in Week 7, in which Microsoft Power BI Desktop was used for the whole process: Power Query for cleaning and transformation, the Model view for defining relationships, and DAX (Data Analysis Expressions) for calculations.

The main case study is the Students Performance dataset, which records the exam scores of 1,000 students together with some background information about each student. A second, shorter exercise used a stock price file for the company SHOP to practise working with date and time columns.

1.1 Objectives

- Load the Students Performance dataset into Power Query and set correct data types.
- Standardise the text columns so that categories are written consistently.
- Create a unique Student id key and a Total marks column.
- Derive Grade and Result (Pass/Fail) columns using conditional logic.
- Split the data into separate tables and unpivot the subject scores.
- Build relationships between the tables in the data model.
- Write DAX measures, including Overall Pass %, in a dedicated Measures table.
- Convert a Date/Time/Timezone column to a plain Date column in the SHOP stock data.

1.2 Tools used

Power Query Editor was used for loading, cleaning and reshaping data; the Model view for relationships; and DAX with Measure tools for calculations stored in a Measures table.

2. Dataset Description

The Students Performance dataset contains 1,000 records, one per student. After the first row was promoted to headers, the table had eight columns, which are listed below.

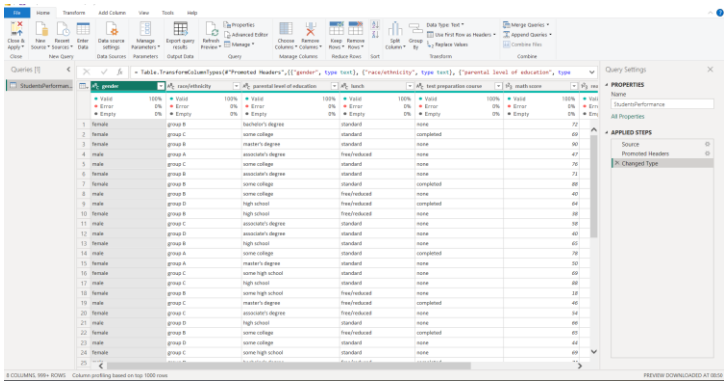
Column	Type	Description
gender	Text	Gender of the student (female / male)
race/ethnicity	Text	Group label (Group A to Group E)
parental level of education	Text	Highest education level of the parents, e.g. bachelor's degree, high school
lunch	Text	Type of lunch: standard or free/reduced
test preparation course	Text	Whether the student completed the preparation course (completed / none)
math score	Whole number	Score in mathematics
reading score	Whole number	Score in reading
writing score	Whole number	Score in writing

Column profiling in Power Query showed 100% valid values, 0% errors and 0% empty values for every column in the preview, so no rows needed to be removed for missing data. The cleaning work therefore concentrated on consistency of text, data types and the creation of new columns.

3. Data Cleaning and Transformation in Power Query

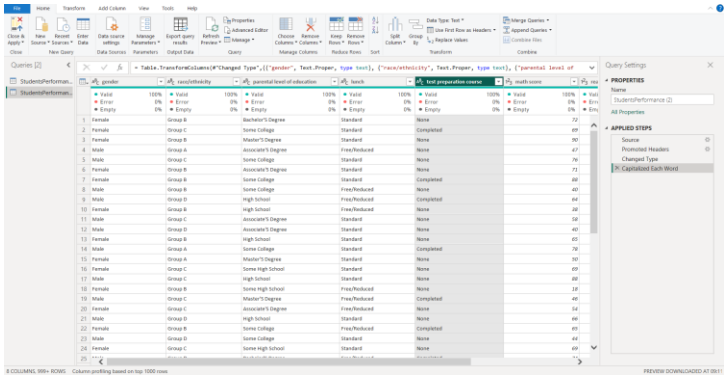
3.1 Promoting headers and setting data types

The file was loaded into the Power Query Editor. The first step, Source, brought in the data, and Promoted Headers turned the first row into column names. Changed Type then assigned a data type to every column: the text columns were set to Text (ABC icon) and the three score columns to Whole Number (123 icon). Correct types are important because they decide which operations are allowed, for example, scores must be numeric before they can be added or averaged.



3.2 Standardising the text

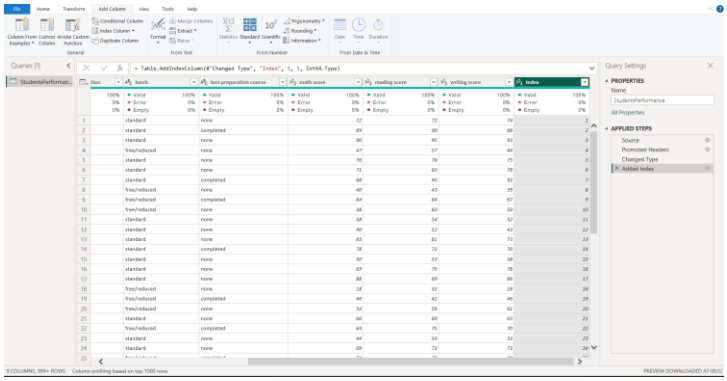
In the raw file the text values were written in lower case, such as "bachelor's degree" and "group b". To make labels look consistent in visuals and slicers, the text columns were converted with Transform > Format > Capitalize Each Word, which applies the Text.Proper function. The step is recorded as Capitalized Each Word, and the result reads "Bachelor's Degree", "Group B", "Free/Reduced" and so on.



The same query also contained steps to trim and clean text, which remove leading and trailing spaces and non-printing characters. This prevents two visually identical values from being treated as different categories.

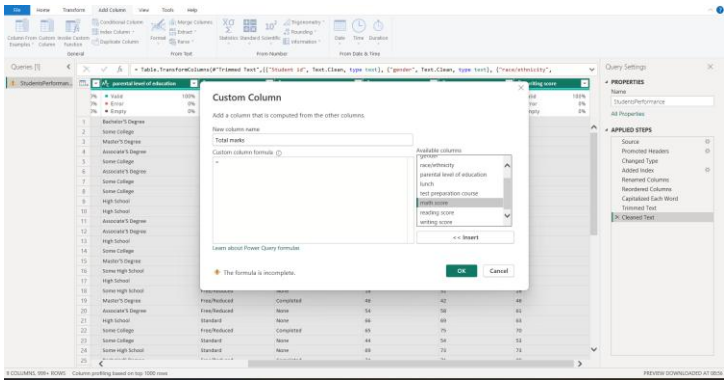
3.3 Adding an index column to create Student id

The dataset has no column that identifies a student, which is a problem when tables are later split and joined. An index column was added with Add Column > Index Column, starting at 1 in the first attempt, and the generated formula was Table.AddIndexColumn(#"Changed Type", "Index", 1, 1, Int64.Type). In the final query an index starting at 0 was used and the column was renamed Student id, so every student has a unique whole-number key.

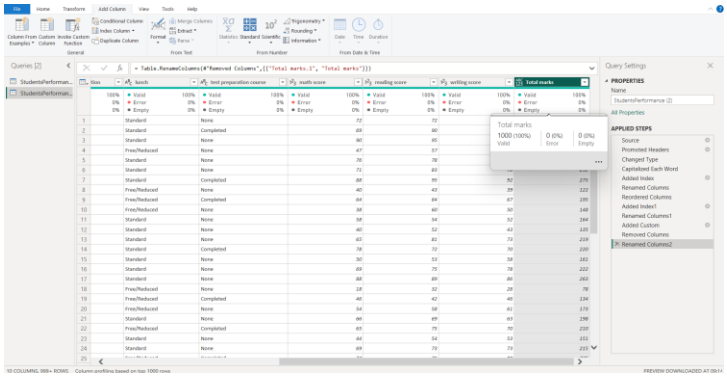


3.4 Creating the Total marks column

A custom column named Total marks was created using Add Column > Custom Column. In the dialog box the new column name is typed, and the formula is built by inserting the columns from the Available columns list. The formula adds the three subject scores: math score + reading score + writing score.



After the column was added, any temporary column was removed and the new column was renamed to Total marks. The preview shows a total for every student, for example 275 for student 7 and 78 for student 18.



3.5 Conditional columns: Grade and Result

Two conditional columns were added through Add Column > Conditional Column. Grade assigns a grade band based on marks, and Result records Pass or Fail. These columns allow the data to be summarised by outcome rather than by raw scores. The applied steps list shows them as Added Conditional Column and Added Conditional Column1.

3.6 Summary of applied steps

The Applied Steps pane records every change so that the whole process can be repeated automatically when the data is refreshed. The main steps of the final query are summarised below.

Step	What it does
Source	Connects to the StudentsPerformance file
Promoted Headers	Uses the first row as column names
Changed Type	Sets Text and Whole Number types
Capitalized Each Word	Applies proper case to text columns
Added Index / Renamed Columns	Creates the Student id key
Reordered Columns	Places Student id first
Added Custom / Removed Columns / Renamed Columns	Creates Total marks and removes helper columns
Added Conditional Column (x2)	Creates Grade and Result
Removed Columns2	Keeps only Student id and the three scores for the marks table
Unpivoted Columns	Turns score columns into Attribute and Value rows

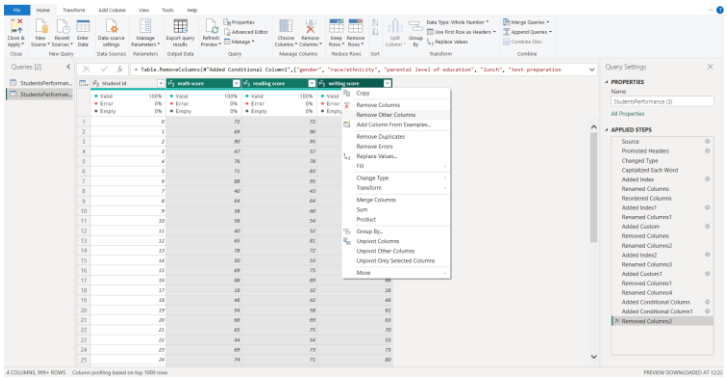
4. Creating the Students Mark Table (Unpivot)

4.1 Why unpivot?

The scores are stored in three separate columns: math score, reading score and writing score. This wide layout is convenient for calculating totals, but it makes it hard to compare subjects in one chart. A long layout, with one row per student per subject, is better suited to charts and slicers.

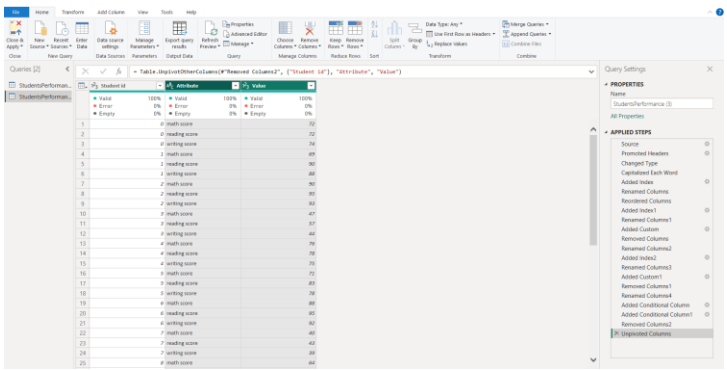
4.2 Selecting the columns to keep

A copy of the query was reduced to the columns needed for the marks table. Using the right-click menu Remove Other Columns after selecting Student id, math score, reading score and writing score, all descriptive columns were removed. The formula bar shows the Table.RemoveColumns call generated by the Removed Columns2 step.



4.3 Applying Unpivot Other Columns

With Student id selected, Transform > Unpivot Other Columns was applied. Power Query generated the formula Table.UnpivotOtherColumns(#"Removed Columns2", {"Student id"}, "Attribute", "Value"). The three score columns became two new columns: Attribute, which holds the subject name, and Value, which holds the score. For example, student 0 now appears in three rows: math score 72, reading score 72 and writing score 74.



This query was named Students Mark. Using Unpivot Other Columns rather than Unpivot Columns means that, if new subjects were added to the source later, they would also be unpivoted without changing the query.

5. Data Model

5.1 Tables in the model

After the queries were loaded with Close & Apply, the model contained three data tables, and later a fourth table for measures.

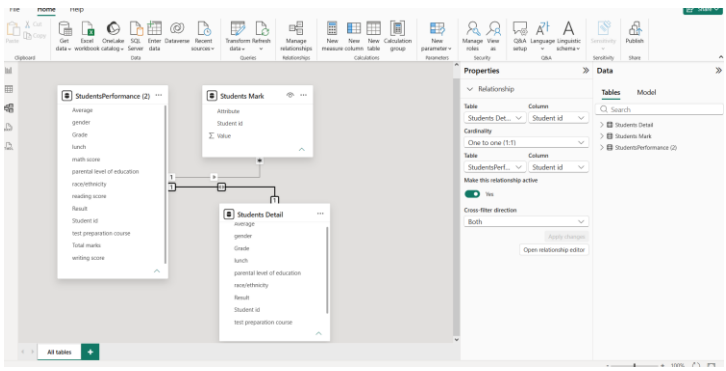
Table	Role	Main columns
StudentsPerformance (2)	Main table with scores and results	Student id, gender, race/ethnicity, parental level of education, lunch, test preparation course, math, reading and writing score, Total marks, Average, Grade, Result
Students Detail	Student attributes	Student id, gender, race/ethnicity, parental level of education, lunch, test preparation course, Average, Grade, Result
Students Mark	Subject scores in long format	Student id, Attribute, Value

Table	Role	Main columns
Measures table	Stores DAX measures	Average math score, Average Reading score, Average writing score, Overall average score, Overall Pass %

5.2 Relationships

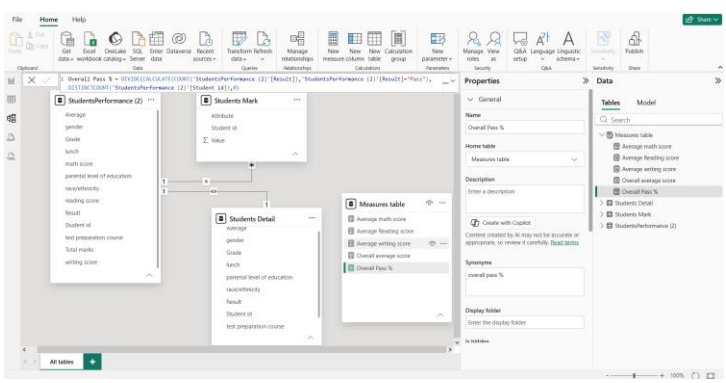
Relationships connect tables through the Student id column so that filters applied to one table flow to the others.

- **Students Detail and StudentsPerformance (2):** a one-to-one (1:1) relationship on Student id. The relationship is active and its cross-filter direction is set to Both.
- **StudentsPerformance (2) and Students Mark:** a one-to-many (1:*) relationship on Student id, because each student has three rows (one per subject) in Students Mark.



5.3 Measures table

A separate table called Measures table was created to hold all measures. Keeping measures in one place makes the Data pane tidy and avoids mixing calculations with data columns. Each measure has its home table set to Measures table in Measure tools.



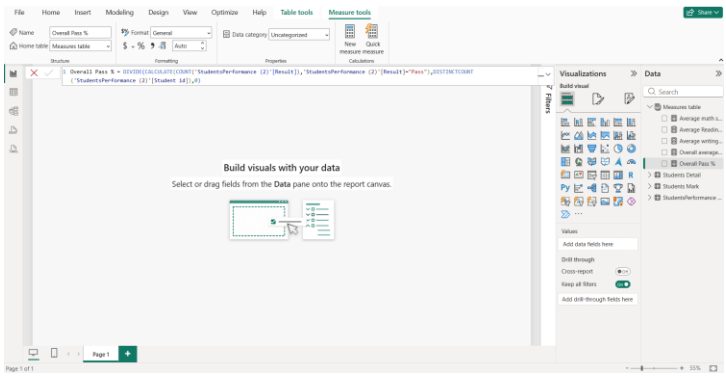
6. DAX Measures

DAX measures are calculated when a visual is drawn, and they respond to filters such as a gender slicer. The following five measures were created in the Measures table.

Measure	Purpose
Average math score	Average of math score for the current filter
Average Reading score	Average of reading score for the current filter
Average writing score	Average of writing score for the current filter
Overall average score	Average across the three subjects
Overall Pass %	Share of students whose Result is Pass

6.1 Overall Pass %

This measure was entered in the formula bar of Measure tools, as shown in the screenshot below.




```
Overall Pass % =
DIVIDE(
    CALCULATE(
        COUNT('StudentsPerformance (2)'[Result]),
        'StudentsPerformance (2)'[Result] = "Pass"
    ),
    DISTINCTCOUNT('StudentsPerformance (2)'[Student id]),
    0
)
```

How the formula works:

- **CALCULATE:** evaluates the expression inside it after applying the extra filter Result = "Pass".
- **COUNT of Result:** counts the rows that remain, which are the students who passed.
- **DISTINCTCOUNT of Student id:** counts the unique students in the current filter context, which is the total number of students.
- **DIVIDE:** divides passed students by total students and returns the alternate result 0 instead of an error when the denominator is zero or blank.

7. Observations, Issues and Improvements

7.1 Observations

- All eight columns of the dataset had 100% valid values, so the quality problems were about formatting, not missing data.

- Unpivoting makes subject comparison easier because Attribute can be used directly as a legend or axis.

7.2 Issues found

- **Total marks data type:** in the preview the column shows the ABC123 (Any) icon. It should be set to Whole Number so it behaves as a number in calculations and visuals.
- **Percentage format:** Overall Pass % currently has the General format, so it would show as a decimal. It should be formatted as a percentage in Measure tools.
- **Duplicate structure:** Students Detail and StudentsPerformance (2) hold many of the same columns and are linked one-to-one. This duplicates data and adds a Both-direction filter that can cause ambiguity.

7.3 Suggested improvements

- Set Total marks to Whole Number and format Overall Pass % as a percentage.
- Keep one table for student attributes and one for scores, or merge Students Detail into the main table, and use single-direction filters where possible.
- Rename the queries to clear names (for example Students Fact) instead of numbered copies such as StudentsPerformance (2).
- Add a date table for the SHOP data and relate it to the new Date column.

8. Learning Outcomes

- Using Power Query to profile, clean and shape data without changing the source file.
- Creating custom and conditional columns, index columns and unpivoted tables.
- Designing a simple model with keys, relationship cardinality and cross-filter direction.
- Writing DAX measures using CALCULATE, COUNT, DISTINCTCOUNT and DIVIDE.

9. Conclusion

In Week 7 the Students Performance dataset was taken from a raw file to an analysis-ready model. Headers and data types were set, text was standardised, a Student id key and Total marks column were created, and Grade and Result columns were derived. The scores were unpivoted into a Students Mark table, three tables were linked on Student id, and a Measures table was created with five DAX measures, including Overall Pass %.

The supplementary SHOP exercise showed how a Date/Time/Timezone column can be converted into a clean Date column. The remaining improvements are small: setting Total marks to Whole Number, formatting Overall Pass % as a percentage and simplifying the one-to-one relationship. The model is now ready for building report pages with cards, bar charts and slicers in the following weeks.